Assignment No. 5

Title

Implement a Java program to demonstrate Single Inheritance and Multilevel Inheritance.

Software / Libraries Required

• Java Development Kit (JDK)
• Any Java IDE (Eclipse / IntelliJ / NetBeans) or Command Prompt

Outcomes of Assignment

After completing this assignment, the student will be able to:
• Understand the concept of inheritance in Java
• Differentiate between single and multilevel inheritance
• Implement inheritance using the extends keyword
• Reuse code using parent and child classes
• Apply object-oriented programming principles effectively

Theory

Java is a class-based, object-oriented programming language that follows the principles of Object-Oriented Programming (OOP) to develop structured, reusable, and maintainable software. The core concepts of OOP include class, object, encapsulation, inheritance, polymorphism, and abstraction. Among these, inheritance plays a significant role in reducing code duplication and improving program organization.

Inheritance is the mechanism by which one class acquires the properties (data members) and behaviors (methods) of another class. The class whose properties are inherited is known as the superclass or parent class, and the class that inherits these properties is known as the subclass or child class.
Inheritance represents an "is-a" relationship between classes. For example, a Student *is a* Person, and a Result *is a* Student. Java achieves inheritance using the extends keyword.

```
class Child extends Parent {
   // class body
}
```

Using inheritance, a child class can access the public and protected members of its parent class, which helps in code reuse and better program structure.

Advantages of Inheritance:

Inheritance provides several benefits in object-oriented programming:

- **Code Reusability**: Common code is written once in the parent class and reused in child classes
- **Reduced Redundancy**: Avoids rewriting the same logic multiple times
- **Improved Maintainability**: Changes made in the parent class are reflected in child classes
- **Logical Hierarchy**: Helps represent real-world relationships
- **Supports Polymorphism**: Forms the foundation for method overriding

Java supports different types of inheritance using classes. In this experiment, the focus is on:

1. **Single Inheritance**
2. **Multilevel Inheritance**

(Java does not support multiple inheritance using classes to avoid ambiguity, but it can be achieved using interfaces.)

Single Inheritance

Single inheritance occurs when a single child class inherits from one parent class.

Structure:

```
Parent Class
    |
 Child Class
```

In single inheritance, the child class automatically gains access to the variables and methods of the parent class. This type of inheritance is simple, easy to understand, and widely used in real-world applications.

Example:

If Student class inherits from Person class, then Student can access the attributes such as name and age defined in Person.

Multilevel Inheritance

Multilevel inheritance occurs when a class is derived from another derived class, forming a chain of inheritance.

Structure:

```
Grandparent Class
      |
  Parent Class
      |
  Child Class
```

In this type of inheritance, the lowest-level class inherits the properties of both the parent and grandparent classes. This allows the program to represent hierarchical relationships more effectively.

Example:
If Result class inherits from Student, and Student inherits from Person, then Result can access the properties of both Student and Person.

Role of Inheritance in Object-Oriented Design
Inheritance helps in building programs that are:

- **Modular**: Each class performs a specific role
- **Scalable**: New features can be added easily
- **Readable**: Code structure is clear and logical
- **Efficient**: Reduces development time

Inheritance is widely used in applications such as student management systems, banking systems, employee records, and enterprise software where hierarchical relationships exist.

Algorithm

1. Start
2. Create a base class Person with attributes name and age
3. Create a derived class Student that extends Person
4. Add roll number and marks in Student class
5. Create another class Result that extends Student
6. Add method to display all details
7. In main method, create object of Result class
8. Call methods to display data
9. Stop

Related Programming Questions

1. Write a program to demonstrate hierarchical inheritance.
2. Create a program using inheritance to model an employee system.
3. Write a program to demonstrate method overriding.
4. Implement inheritance using constructors.

Oral Questions

1. What is inheritance in Java?
2. What is single inheritance?
3. What is multilevel inheritance?
4. Which keyword is used to achieve inheritance in Java?
5. Why does Java not support multiple inheritance using classes?
6. What is a parent class in Java?
7. What is a child class in Java?
8. What is the meaning of the “is-a” relationship in inheritance?

9. What are the advantages of inheritance?
10. Can a child class access private members of a parent class?
11. What types of inheritance are supported by Java using classes?
12. How does inheritance help in code reusability?
13. What is hierarchical inheritance?

Conclusion

Assignment No. 6

Title

Develop a Java program using interfaces to calculate the area of different shapes.

Software / Libraries Required

• Java Development Kit (JDK)
• Any Java IDE (Eclipse / IntelliJ / NetBeans) or Command Prompt

Outcomes of Assignment

After completing this assignment, the student will be able to:
• Understand the concept of interfaces in Java
• Implement interfaces using the implements keyword
• Achieve abstraction using interfaces
• Calculate area of different shapes using polymorphism
• Apply object-oriented programming principles effectively

Theory

Java is a fully object-oriented programming language that supports key OOP principles such as encapsulation, inheritance, polymorphism, and abstraction. Among these, abstraction helps in hiding implementation details and exposing only essential features to the user. One of the most important ways to achieve abstraction in Java is through interfaces.

An interface in Java is a collection of abstract methods and constants. It defines *what* a class must do, but not *how* it should do it. Interfaces are used to achieve 100% abstraction, as all methods declared in an interface are abstract by default (before Java 8).
Syntax of an interface:

```
interface InterfaceName {
   void methodName();
}
```

A class implements an interface using the implements keyword.

```
class ClassName implements InterfaceName {
   // method implementation
}
```

Importance of Interfaces:

Interfaces play a crucial role in Java programming because:

- They support abstraction

- They allow multiple inheritance
- They improve **flexibility and scalability**
- They promote **loose coupling**
- They make programs easier to maintain

Interface	Class
Supports full abstraction	Supports partial abstraction
Methods are abstract	Methods may have implementation
Uses implements keyword	Uses extends keyword
Multiple interfaces allowed	Multiple inheritance not allowed

Real-World Use of Interfaces:
Interfaces are commonly used in:

- Banking systems
- Shape and geometry calculations
- Device drivers
- Enterprise applications
- Framework design

In real-world scenarios, different objects may perform the same action differently. Interfaces help define a **common behavior** for such objects.

Advantages of Using Interfaces:
• Provides complete abstraction
• Encourages modular programming
• Improves code reusability
• Makes applications scalable
• Supports polymorphism

Role of Polymorphism:
Polymorphism allows an interface reference to refer to different implementing objects. The method call is resolved at **runtime**, depending on the object type. This makes programs flexible and extensible.

Algorithm

1. Start
2. Create an interface Shape with method calculateArea()
3. Create classes Circle, Rectangle, and Triangle implementing Shape
4. Define area calculation logic in each class

5. In main method, create objects using interface reference
6. Call calculateArea() method
7. Display the results
8. Stop

Related Programming Questions

1. Write a program to demonstrate multiple inheritance using interfaces.
2. Create an interface for banking operations and implement it.
3. Write a program using interface to calculate perimeter of shapes.

Oral Questions

1. What is an interface in Java?
2. How is an interface different from a class?
3. Which keyword is used to implement an interface in Java?
4. Can a class implement more than one interface? Why?
5. Why are interfaces used in Java?
6. Can an interface contain variables? If yes, what is their nature?
7. Are methods in an interface abstract or concrete?
8. Can an interface have a constructor? Why or why not?
9. What access modifier is used by default for methods in an interface?
10. Can an interface extend another interface?
11. What is the difference between `extends` and `implements` keywords?
12. Can we create an object of an interface? Explain.
13. What happens if a class does not implement all methods of an interface?
14. How do interfaces support abstraction in Java?
15. Give one real-time example where interfaces are used.

Conclusion

Assignment No. 7

Title

Write a Java program to demonstrate exception handling for division by zero and invalid input.

Software / Libraries Required

• Java Development Kit (JDK)
• Any Java IDE (Eclipse / IntelliJ / NetBeans) or Command Prompt

Outcomes of Assignment

After completing this assignment, the student will be able to:
• Understand the concept of exceptions in Java
• Handle runtime errors using try, catch, and finally blocks
• Handle division by zero using ArithmeticException
• Handle invalid user input using InputMismatchException
• Develop robust and error-free Java programs

Theory

Java is a **robust and reliable object-oriented programming language** that provides strong support for **exception handling**. During program execution, errors may occur due to invalid input, logical mistakes, or unexpected conditions. If these errors are not handled properly, they may cause abnormal termination of the program. Exception handling allows programmers to **detect errors and handle them gracefully**, ensuring smooth execution of programs.
An **exception** is an abnormal condition or unexpected event that occurs during the execution of a program and disrupts its normal flow. Examples include division by zero, accessing invalid array indexes, or entering incorrect input.
Java provides a built-in **exception-handling mechanism** to manage such runtime errors effectively.

Types of Errors in Java

1. **Compile-time Errors**
   - Detected during compilation
   - Example: syntax errors
2. **Runtime Errors (Exceptions)**
   - Occur during program execution
   - Example: division by zero
3. **Logical Errors**
   - Program runs but produces incorrect output

This experiment focuses on **runtime errors**, also known as **exceptions**.

Exception Handling Mechanism
Java uses the following keywords for exception handling:

- **try** – Contains code that may cause an exception
- **catch** – Handles the exception
- **finally** – Executes code whether an exception occurs or not
- **throw** – Used to explicitly throw an exception
- **throws** – Declares exceptions

try–catch Block
The try block encloses statements that may cause an exception. If an exception occurs, the control is transferred to the corresponding catch block.

```
try {
   // risky code
} catch (Exception e) {
   // handling code
}
```

Division by Zero Exception
When a number is divided by zero in Java, an **ArithmeticException** occurs. This is a runtime exception. Handling this exception prevents the program from crashing and allows it to display a user-friendly message.
Example:

```
int result = a / b; // b = 0 causes ArithmeticException
```

Invalid Input Exception
When a user enters input of the wrong data type (for example, entering text instead of an integer), Java throws an **InputMismatchException**. This exception commonly occurs when using the Scanner class for input.
Handling this exception ensures that the program responds gracefully to incorrect user input.

Exception handling provides several benefits:

- Prevents abnormal program termination
- Improves program reliability
- Makes programs user-friendly
- Simplifies debugging and maintenance
- Helps in writing secure and robust code

Real-World Applications:

Exception handling is widely used in:

- Banking and financial applications
- Online transaction systems
- User input validation
- Database connectivity programs
- Enterprise-level software

Algorithm

1. Start
2. Accept two numbers from the user
3. Perform division operation
4. Use try block to enclose risky code
5. Catch ArithmeticException for division by zero
6. Catch InputMismatchException for invalid input
7. Display appropriate error messages
8. Execute finally block
9. Stop

Related Programming Questions

1. Write a program to demonstrate multiple catch blocks.
2. Write a program using throw keyword.
3. Demonstrate user-defined exceptions in Java.

Oral Questions

1. What is an exception in Java?
2. What is the difference between an error and an exception?
3. What is exception handling? Why is it needed?
4. Which keywords are used for exception handling in Java?
5. What is the purpose of the `try` block?
6. What happens if an exception is not handled in a Java program?
7. What is `ArithmeticException`? When does it occur?
8. What is `InputMismatchException`?
9. Can multiple `catch` blocks be used with a single `try` block?
10. What is the use of the `finally` block?
11. Is the `finally` block always executed?
12. What happens if an exception occurs inside the `finally` block?
13. Can we write a program without a `catch` block? If yes, how?

14. What is the difference between `throw` and `throws`?
15. Give one real-time example where exception handling is required.

Conclusion

Assignment No. 8

Title

Create a program to simulate a banking system with exception handling for invalid transactions.

Software / Libraries Required

• Java Development Kit (JDK)
• Any Java IDE (Eclipse / IntelliJ / NetBeans) or Command Prompt

Outcomes of Assignment

After completing this assignment, the student will be able to:
• Understand the concept of exceptions in Java
• Handle runtime errors using try, catch, and finally blocks
• Handle invalid transactions such as insufficient balance or negative amount
• Handle invalid input using InputMismatchException
• Develop robust and error-free Java programs for real-world scenarios

Theory

Java is a robust, object-oriented programming language that supports exception handling, allowing programs to deal with unexpected conditions at runtime. In a banking system, invalid transactions such as attempting to withdraw more than the account balance or entering negative amounts can occur. Exception handling allows the program to handle these conditions gracefully without crashing.

An exception is an abnormal condition that occurs during program execution and disrupts the normal flow. Examples in banking include:

- Withdrawal amount exceeding balance
- Entering a negative amount
- Entering invalid input such as text instead of a number

Java provides a built-in mechanism to handle exceptions using keywords such as try, catch, finally, throw, and throws.

Types of Errors in Java

1. **Compile-time Errors**
   - Detected during compilation
   - Example: syntax errors
2. **Runtime Errors (Exceptions)**
   - Occur during execution
   - Example: division by zero, invalid withdrawal

3. **Logical Errors**
    - Program runs but produces incorrect output

This program focuses on **runtime errors**, also known as exceptions.

Exception Handling Mechanism

- **try** – Contains code that may cause an exception
- **catch** – Handles the exception
- **finally** – Executes code whether an exception occurs or not
- **throw** – Used to explicitly throw an exception
- **throws** – Declares exceptions

Example of try-catch block:

```
try {
   // risky code
} catch (Exception e) {
   // handling code
}
```

Invalid Transaction Handling

In this banking system program, exceptions are used to handle:

1. **InsufficientBalanceException** – Raised when withdrawal amount exceeds account balance
2. **NegativeAmountException** – Raised when a negative deposit or withdrawal is entered
3. **InputMismatchException** – Raised when the user enters invalid input

This ensures that the program **does not crash** and gives user-friendly messages.

Importance of Exception Handling in Banking Systems

- Prevents program from crashing due to invalid transactions
- Ensures data integrity
- Improves user experience with meaningful messages
- Allows robust and secure application design
- Helps in handling real-world scenarios efficiently

Real-World Applications

Exception handling is widely used in:

- Banking and financial applications
- Online transaction systems
- ATM software
- User input validation
- Enterprise-level applications

Algorithm

1. Start
2. Initialize account balance
3. Display banking menu (Deposit, Withdraw, Check Balance, Exit)
4. Accept user choice
5. Use try block to perform transaction
6. Catch **InsufficientBalanceException** for withdrawal exceeding balance
7. Catch **NegativeAmountException** for negative deposits or withdrawals
8. Catch **InputMismatchException** for invalid input
9. Display appropriate messages
10. Execute finally block
11. Repeat until user chooses to exit
12. Stop

Related Programming Questions

1. Write a program to handle invalid array index using exception handling.
2. Write a program to simulate ATM operations with multiple exceptions.
3. Write a program to handle file not found exception.

Oral Questions

1. What is an exception in Java?
2. What is exception handling? Why is it important in banking systems?
3. Which keywords are used for exception handling in Java?
4. What is the purpose of the try block?
5. What is the difference between throw and throws?
6. Can multiple catch blocks be used with a single try block?
7. What is a custom (user-defined) exception?
8. How is InputMismatchException handled?
9. What happens if a negative amount is entered in a banking transaction?
10. What happens if withdrawal amount exceeds balance?
11. What is the purpose of the finally block?
12. Can a program continue execution after an exception occurs?
13. Why is exception handling important in real-world applications?

Conclusion

Assignment No. 9

Title

Write a Java applet to display a welcome message and change the background colour dynamically.

Software / Libraries Required

• Java Development Kit (JDK)
• Any Java IDE (Eclipse / IntelliJ / NetBeans) or Command Prompt
• Web browser / Applet viewer for running applets

Outcomes of Assignment

After completing this assignment, the student will be able to:
• Understand the concept of Java Applets
• Learn how to create and run an applet program
• Display text messages in an applet window
• Change background colors dynamically using applet methods
• Apply object-oriented programming concepts in GUI development

Theory

Java Applets are special types of Java programs that are designed to be embedded in web pages and executed by a browser or an applet viewer. Unlike standard Java applications, which run independently, applets run within the security constraints of a web browser, making them suitable for creating interactive and dynamic web content. Applets are subclasses of the java.applet.Applet class, and they provide a graphical interface to interact with users.

Applets are widely used to display text, images, and animations, respond to user events like mouse clicks and keyboard input, and dynamically update content based on user interaction or time-based changes. For example, applets can be used to create interactive advertisements, games, or dynamic web-based tools such as calculators or clocks.

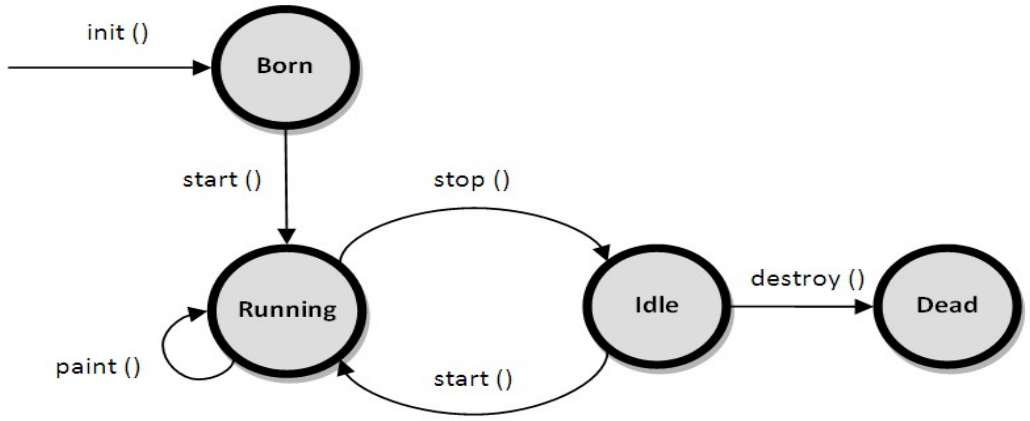
Key Features of Applets

1. **Lightweight GUI components** – Applets are smaller and faster than applications with full window frames.
2. **Graphics support** – Applets can draw text, shapes, and images using the Graphics class.
3. **Event handling** – Applets can respond to user actions such as mouse clicks or keyboard events.
4. **Dynamic behavior** – Applets can change their appearance, background color, or content at runtime.

Lifecycle of an Applet

The lifecycle of a Java applet is managed by the browser and includes the following methods:

1. init() – Called once when the applet is loaded. Used for initialization.
2. start() – Called when the applet starts or resumes.
3. paint(Graphics g) – Called to display content in the applet window.
4. stop() – Called when the applet is stopped or hidden.
5. destroy() – Called when the applet is terminated.



Changing Background Dynamically

- The setBackground(Color c) method is used to change the background color of the applet.
- Colors can be set using predefined constants in the Color class (e.g., Color.RED) or by using RGB values (new Color(r, g, b)).
- Using a timer or user interaction, the background color can be changed dynamically.

Displaying Text in Applet

- The drawString(String str, int x, int y) method of the Graphics class is used to display text.
- Coordinates (x, y) specify the position of the text in the applet window.

Advantages of Applets

• Lightweight and portable
• Can run on any platform with a Java-enabled browser
• Interactive and visually appealing
• Supports graphics and animation

Algorithm

1. Start
2. Import java.applet.* and java.awt.* packages
3. Create a class extending Applet
4. Override init() to set initial applet parameters
5. Override paint(Graphics g) to display the welcome message
6. Use setBackground(Color c) to change background dynamically
7. Optionally, change background color based on a timer or user action

8. Run the applet in a browser or applet viewer
9. Stop / destroy applet when user exits
10. End

HTML Code to Run Applet

```
<html>
  <body>
    <applet code="WelcomeApplet.class" width="400" height="200">
    </applet>
  </body>
</html>
```

Related Programming Questions

1. Write an applet to display multiple messages in different colors.
2. Write an applet to draw shapes (rectangle, circle, oval) dynamically.
3. Create an applet to simulate traffic light changes.

Oral Questions

1. What is an applet in Java?
2. What is the difference between an applet and an application?
3. Which class must be extended to create an applet?
4. Name the lifecycle methods of a Java applet.
5. Which method is used to display graphics in an applet?
6. How do you change the background color of an applet?
7. Can an applet interact with the user? How?
8. What is the use of the Graphics class in applets?
9. How is a Java applet run in a browser?
10. What are the advantages of using applets?
11. Can we use setForeground() to change text color?
12. How can you make an applet dynamic or interactive?
13. What is the difference between repaint() and paint() in an applet?
14. Can applets handle events like mouse clicks?
15. Give one real-life application of Java applets.

Conclusion

Assignment No. 10

Title

Create a GUI-based calculator using Swing components like buttons and text fields.

Software / Libraries Required

• Java Development Kit (JDK)
• Any Java IDE (Eclipse / IntelliJ / NetBeans) or Command Prompt

Outcomes of Assignment

After completing this assignment, the student will be able to:
• Understand the concept of Java Swing for GUI programming
• Create GUI components like JFrame, JButton, JTextField, and JPanel
• Implement event handling using ActionListener
• Develop a basic calculator to perform arithmetic operations
• Apply object-oriented programming principles in GUI development

Theory

Java provides Swing, which is a part of the Java Foundation Classes (JFC), for building graphical user interfaces (GUIs). Swing allows developers to create rich, interactive, and platform-independent desktop applications. Unlike AWT (Abstract Window Toolkit), which relies on native system resources, Swing components are lightweight, meaning they are written entirely in Java and can run consistently on any platform.
Swing provides a wide range of GUI components, including windows, buttons, text fields, labels, panels, checkboxes, radio buttons, tables, and menus, allowing developers to build professional and interactive user interfaces. Swing also supports event-driven programming, which enables applications to respond to user actions such as mouse clicks, keyboard input, or other events.
A calculator GUI is a common example used to teach Swing because it involves displaying input/output, handling multiple buttons, performing operations, and updating results dynamically. It helps students understand the integration of GUI components with program logic and event handling.

Main Swing Components Used in Calculator

1. **JFrame**
    - The **JFrame** is a **top-level window** that acts as the main container for all GUI components.
    - It provides the **title bar, minimize/maximize buttons, and close button**, and serves as the primary window where the user interacts with the application.

- Example: JFrame frame = new JFrame("Calculator");

2. **JButton**
   - **JButton** is used to create clickable buttons in the GUI.
   - In a calculator, buttons represent **digits (0–9)**, **operators (+, -, *, /)**, and **commands (C for clear, = for equals)**.
   - Buttons generate **action events** when clicked, which are handled by event listeners.
   - Example: JButton addButton = new JButton("+");
3. **JTextField**
   - **JTextField** is used to **display input and results** in a single-line editable or non-editable text box.
   - In calculators, it shows the **numbers entered by the user and the results of operations**.
   - Example: JTextField textField = new JTextField();
4. **JPanel**
   - **JPanel** is a **container used to group related components** and organize them in a specific layout.
   - Panels are often combined with **layout managers** like GridLayout or FlowLayout to arrange buttons in rows and columns.
   - Example: JPanel panel = new JPanel(new GridLayout(4, 4));

Event handling allows the calculator to respond to user actions, such as clicking a button to enter a digit or perform an arithmetic operation.

- To handle events, the ActionListener interface is implemented in the calculator class.
- The actionPerformed() method is overridden to define the actions for each button.
- Key methods and points include:
  - addActionListener(this) – attaches the listener to a button so it can respond to clicks.
  - getActionCommand() – retrieves the text label of the button clicked, which helps identify whether the user clicked a number, operator, or command.
  - The calculator logic processes the input and performs addition, subtraction, multiplication, or division based on the operator selected.

Example flow:

1. User clicks a digit button → digit appears in the text field.
2. User clicks an operator (+, -, *, /) → the current number is stored, and the operator is recorded.
3. User clicks another digit → displayed in text field.
4. User clicks = → operation is performed, and the result is shown in the text field.
5. User clicks C → text field is cleared.

Advantages of Using Swing

1. **Lightweight and Platform-Independent**

- Swing components are written entirely in Java, ensuring consistent behavior across platforms.

2. **Rich GUI Components**
   - Swing provides a wide variety of components to build professional-looking interfaces, including tables, trees, sliders, and progress bars.
3. **Event-Driven Programming**
   - Event handling allows applications to respond dynamically to user actions, making the programs interactive and user-friendly.
4. **Ease of GUI Creation**
   - Swing supports **layouts and panels**, which simplify arranging components and designing complex interfaces like a calculator.
5. **Customizable Appearance**
   - Components' colors, fonts, and sizes can be easily customized, giving flexibility in designing intuitive and attractive GUIs.

Real-World Applications of Swing

- Desktop applications such as text editors, calculators, and media players
- Banking and financial software requiring input fields and buttons
- Educational tools and simulations for interactive learning
- Administrative software with forms, tables, and interactive dashboards

Algorithm

1. Start
2. Import necessary Swing and AWT packages
3. Create a class extending JFrame and implement ActionListener
4. Initialize JTextField to display input and results
5. Initialize JButton for digits (0–9) and operators (+, -, *, /, =, C)
6. Add buttons and text field to the frame using JPanel and layout manager
7. Attach ActionListener to all buttons
8. In actionPerformed(), perform operations based on button clicked
9. Display result in the text field
10. Run the program
11. Stop

Related Programming Questions

1. Create a GUI-based scientific calculator with additional functions like sin, cos, sqrt.
2. Write a program to create a calculator using JTextArea instead of JTextField.
3. Implement a calculator GUI with keyboard input support.

Oral Questions

1. What is Swing in Java?
2. What is the difference between AWT and Swing?
3. What is the purpose of JFrame in Swing?
4. What is JButton used for?
5. What is JTextField?
6. How do you add action events to buttons?
7. What is the purpose of ActionListener?
8. How do you get the text entered in a text field?
9. What is the role of setLayout() in a JPanel?
10. How do you handle multiple buttons in one actionPerformed method?
11. What is the purpose of setVisible(true)?
12. Can Swing components be customized (color, font, size)?
13. How do you prevent a text field from being editable?
14. What happens if division by zero is performed in the calculator?

Conclusion

Assignment No. 11

Title

Write a Java program to demonstrate the Singleton Design Pattern and explain its real-world use cases.

Software / Libraries Required

• Java Development Kit (JDK)
• Any Java IDE (Eclipse / IntelliJ / NetBeans) or Command Prompt

Outcomes of Assignment

After completing this assignment, the student will be able to:
• Understand the concept of **design patterns** in Java
• Learn the **Singleton design pattern** and its implementation
• Ensure only **one instance of a class** is created during runtime
• Apply Singleton pattern in real-world applications like logging, configuration, or database connections
• Enhance object-oriented programming skills by using **static methods and private constructors**

Theory

Design patterns are **reusable solutions** to common software design problems. They act as **templates** that can be applied in various contexts to improve software quality, maintainability, and scalability. Design patterns are classified into three categories:

1. **Creational Patterns** – Focus on **object creation**. Examples: Singleton, Factory, Builder.
2. **Structural Patterns** – Focus on **object composition**. Examples: Adapter, Decorator, Composite.
3. **Behavioral Patterns** – Focus on **communication between objects**. Examples: Observer, Strategy, Command.

The **Singleton pattern** belongs to **creational patterns**, ensuring that only **one object of a class** is created and providing a **global point of access** to that object. The Singleton pattern restricts the instantiation of a class to **one single instance**. This is particularly useful when **exactly one object** is needed to coordinate actions across the system.

Key characteristics:

- A **single instance** exists in the application.
- The constructor is **private** to prevent instantiation from outside the class.
- A **static method** (commonly getInstance()) provides global access to the instance.

Advantages:

- Controlled access to the sole instance.

- Saves memory by preventing multiple object creation.
- Centralized resource management.
- Simplifies **global configuration management**.

Disadvantages:

- Can make unit testing harder due to **global state**.
- Can be misused, leading to tight coupling.
- Multithreaded applications may require **synchronization**, which can affect performance.

Real-World Analogy

Think of a **president of a country**. There is only **one president at a time**, and all government decisions pass through this single authority. Similarly, the Singleton pattern ensures that all parts of an application **use the same instance** for shared responsibilities.

Steps to implement Singleton:

1. Create a private static variable to hold the single instance.
2. Make the constructor private to prevent instantiation from outside the class.
3. Create a public static method (commonly getInstance()) that returns the instance.

Example: Lazy Initialization Singleton

```
class Singleton {
   private static Singleton singleInstance = null; // Single instance

   private Singleton() { // Private constructor
      System.out.println("Singleton object created.");
   }

   public static Singleton getInstance() {
      if (singleInstance == null) { // Create instance if not exists
         singleInstance = new Singleton();
      }
      return singleInstance;
   }

   public void showMessage() {
      System.out.println("This is the Singleton instance.");
   }
}
```

Types of Singleton Pattern

1. **Lazy Initialization Singleton**
   - Instance is created only when required.
   - Saves memory but is not thread-safe by default.
2. **Eager Initialization Singleton**

- Instance is created at class loading time.
- Thread-safe by default but may waste memory if instance is never used.

3. private static Singleton instance = new Singleton();
4. **Thread-Safe Singleton**
   - Ensures safe execution in multithreaded environments using synchronized.

Event-Driven Usage of Singleton

Singleton is commonly used in centralized control scenarios:

- **Logging systems:** One logger instance writes all logs, ensuring consistency.
- **Configuration management:** One instance holds all app settings globally.
- **Database connections:** One connection pool instance manages all connections.
- **Caching systems:** Singleton cache avoids redundant memory usage.

Example: A logger writes all program messages to a file. If multiple instances were allowed, logs could be inconsistent or overwritten. Singleton ensures single source of truth.

Real-World Use Cases of Singleton Pattern

1. **Logging** – Maintain a single log file by using one instance of a logger.
2. **Configuration Management** – Store application settings in a single instance.
3. **Database Connections** – Ensure only one connection manager exists for efficiency.
4. **Thread Pooling** – Manage resources by creating a single pool instance.
5. **Cache Management** – Maintain a single cache instance to store frequently accessed data.

Algorithm

1. Start
2. Create a class with a **private static instance** variable
3. Make the constructor **private**
4. Create a **public static method** to return the instance
5. Access the instance and call class methods
6. Verify that all references point to the **same object**
7. Stop

Related Programming Questions

1. Implement Thread-Safe Singleton using synchronized method.
2. Demonstrate Eager Initialization Singleton in Java.
3. Create a Logger class using Singleton pattern.

Oral Questions

1. What is a design pattern?
2. What is the Singleton design pattern?
3. Why is the constructor in a Singleton class private?
4. How do you ensure only one instance is created?

5. What is the difference between eager and lazy initialization?
6. Can Singleton be made thread-safe? How?
7. Give real-world applications of the Singleton pattern.
8. What is the difference between Singleton and static classes?
9. Can Singleton be inherited?
10. How is memory saved by using Singleton?

Conclusion

Assignment No. 12

Title

Develop a Java program to create multiple threads for tasks like file reading and mathematical computations, ensuring thread synchronization.

Software / Libraries Required

- Java Development Kit (JDK)
- Any Java IDE (Eclipse / IntelliJ / NetBeans) or Command Prompt

Outcomes of Assignment

After completing this assignment, the student will be able to:

- Understand the concept of multithreading in Java
- Create and execute multiple threads for simultaneous tasks
- Implement thread synchronization to prevent data inconsistency
- Handle tasks like file reading and mathematical computations concurrently
- Apply thread-safe programming practices to avoid race conditions

Theory

Multithreading is a powerful feature of Java that allows a program to execute multiple tasks concurrently. Each task runs as a thread, which is a lightweight sub-process of the main program. Unlike separate processes, threads share memory and resources of the parent process, making communication between them more efficient.

Multithreading enables developers to divide complex programs into smaller, parallel tasks, improving performance, responsiveness, and resource utilization.

Benefits of Multithreading

1. Improved Program Performance: Threads allow tasks to run in parallel. For example, while one thread reads data from a file, another thread can perform computations.
2. Better Resource Utilization: Idle CPU time is reduced because multiple threads can execute simultaneously.
3. Responsiveness: In GUI-based applications, multithreading keeps the interface responsive while background tasks run.
4. Modularity: Large programs can be divided into independent threads performing specific tasks.

Example: In a banking system, one thread can handle deposits, another can handle withdrawals, and another can generate statements concurrently.

Creating Threads in Java

Java provides two ways to create threads:

1. By Extending the Thread Class
   - Create a subclass of Thread.
   - Override the run() method to define the task.
   - Call start() to begin execution.

```
class MyThread extends Thread {
   public void run() {
      System.out.println("Thread running: " + Thread.currentThread().getName());
   }
}
```

2. By Implementing the Runnable Interface
   - Implement the Runnable interface and define the run() method.
   - Create a Thread object and pass the Runnable instance to it.
   - Call start() to execute the thread.

```
class MyTask implements Runnable {
   public void run() {
      System.out.println("Runnable thread running: " + Thread.currentThread().getName());
   }
}
```

Difference Between Thread and Runnable:

Feature	Thread Class	Runnable Interface
Inheritance	Extends Thread	Implements Runnable
Multiple Inheritance	Not possible	Possible (can extend other classes)
Flexibility	Less flexible	More flexible

Thread Synchronization

When multiple threads share a common resource (like a file or variable), there is a risk of data inconsistency, also called a race condition.

Synchronization ensures that only one thread accesses the shared resource at a time, preventing conflicts.

- In Java, this is achieved using the synchronized keyword.
- Synchronization can be applied to methods or blocks of code.

Example of Synchronized Method:

```
class SharedResource {
   synchronized void accessResource(String taskName) {
      for (int i = 1; i <= 5; i++) {
         System.out.println(taskName + " - Step " + i + " by " +
Thread.currentThread().getName());
         try { Thread.sleep(500); } catch (InterruptedException e) {}
      }
```

```
    }
}
```

- Only one thread can execute accessResource() at a time.
- Other threads must wait until the resource becomes available.

Thread Life Cycle

A thread goes through five states in its life cycle:

1. New (Created) – Thread object is created but not started.
2. Runnable – Thread is ready to run and waiting for CPU scheduling.
3. Running – Thread is currently executing.
4. Waiting/Blocked – Thread is waiting for a resource or event.
5. Terminated – Thread has finished execution.

Methods Managing Thread Life Cycle:

- start() – Moves thread from New to Runnable.
- run() – Contains the code to be executed by the thread.
- sleep() – Pauses execution temporarily.
- join() – Waits for another thread to complete.
- yield() – Suggests the thread scheduler to give CPU to other threads.

Applications of Multithreading

1. File Handling: Read and write large files simultaneously.
2. Mathematical Computation: Perform multiple calculations in parallel for faster results.
3. Web Servers: Handle multiple client requests simultaneously.
4. Gaming: Update game state, graphics, and user input concurrently.
5. Banking Systems: Process deposits, withdrawals, and balance checks concurrently.
6. Multimedia Applications: Play audio/video while updating GUI or downloading files.

Advantages of Multithreading

- Concurrent execution of tasks increases performance.
- Efficient CPU utilization reduces idle time.
- Responsiveness ensures applications remain interactive.
- Safe access to shared resources using synchronized methods.
- Reduced program execution time for complex tasks.

Challenges in Multithreading

- Race Conditions: Multiple threads modifying shared data simultaneously.
- Deadlocks: Two or more threads waiting indefinitely for resources held by each other.
- Thread Starvation: Low-priority threads may never get CPU time.
- Complexity: Debugging multithreaded programs is harder than sequential programs.

Algorithm

1. Start
2. Define tasks: File reading and mathematical computations

3. Implement Runnable interface for each task
4. Create shared resource object
5. Use synchronized methods or blocks to prevent conflicts
6. Create threads and start execution using start()
7. Threads execute tasks concurrently while synchronized access prevents conflicts
8. Stop

Related Programming Questions

1. Implement multiple threads to print even and odd numbers with synchronization.
2. Create a bank account program with deposit and withdrawal threads.
3. Write a program with threads reading and writing from/to a shared file.
4. Develop a producer-consumer problem with thread synchronization.

Oral Questions

1. What is a thread in Java?
2. How many ways can threads be created in Java?
3. What is the difference between extending Thread and implementing Runnable?
4. What is thread synchronization?
5. How does the synchronized keyword work?
6. What is a race condition?
7. How does Java ensure safe access to shared resources?
8. Can a thread be restarted after completing execution?
9. What is the difference between start() and run() methods?
10. Give examples of real-world applications using threads.

Conclusion